Matrix Operations Using Dynamic Memory Allocation

Addition, Subtraction and Multiplication using Pointer Notation

Name	-	Rudra Pratap Singh
SAP ID	-	590032811
Batch	-	16

1. Problem Statement

Write a C program to dynamically allocate two matrices and perform addition, subtraction and multiplication using pointer notation. Each operation must be implemented as a separate function, and all allocated memory must be released properly before the program ends.

2. Objective

- To understand dynamic memory allocation using malloc(), calloc() and free().
- To use pointers and pointer-to-pointer variables to represent a two-dimensional matrix.
- To access matrix elements with pointer notation (*(*(m + i) + j)) instead of array subscripts.
- To write modular code, with one function for every operation.
- To avoid memory leaks by releasing every block that was allocated.

3. Concepts Used

3.1 Dynamic Matrix Using int **

A matrix with r rows and c columns is created in two steps. First, an array of r pointers is allocated (one pointer for each row). Then, for every row pointer, an array of c integers is allocated. The variable that holds the matrix is therefore of type int ** (pointer to pointer to int).

3.2 Pointer Notation

For a matrix `m`, the element at row `i` and column `j` is accessed as follows:

- `*(m + i)` gives the pointer to the i-th row.
- `*(m + i) + j` gives the address of the element at (i, j).
- `*(*(m + i) + j)` gives the value of the element at (i, j); it is equivalent to `m[i][j]`.

3.3 Rules for the Operations

- **Addition and Subtraction:** both matrices must have the same order (r1 = r2 and c1 = c2). The result has the same order.
- **Multiplication:** the number of columns of A must equal the number of rows of B (c1 = r2). The result has order r1 x c2, and each element is `C[i][j] = sum of A[i][k] * B[k][j]`.

4. Algorithm

1. Start.
2. Read the order (rows and columns) of Matrix A and Matrix B.

3. Dynamically allocate memory for both matrices using allocateMatrix().
4. Read the elements of Matrix A and Matrix B using readMatrix().
5. Display both matrices.
6. If the orders of A and B are equal, call addMatrices() and subtractMatrices() and display the results; otherwise print an error message.
7. If the number of columns of A equals the number of rows of B, call multiplyMatrices() and display the result; otherwise print an error message.
8. Release the memory of every matrix using freeMatrix().
9. Stop.

5. Functions Used

Function	Purpose
`allocateMatrix()`	Allocates memory for a matrix of the given order (row pointers with malloc, rows with calloc). Returns NULL on failure and frees any partly allocated memory.
`readMatrix()`	Reads the elements of a matrix from the keyboard using pointer notation.
`displayMatrix()`	Prints a matrix in row and column format with a title.
`addMatrices()`	Allocates a result matrix and stores A + B in it.
`subtractMatrices()`	Allocates a result matrix and stores A - B in it.
`multiplyMatrices()`	Allocates a result matrix of order r1 x c2 and stores A x B in it.
`freeMatrix()`	Frees every row, then the array of row pointers, so no memory leaks.

6. Source Code

File name: `matrix_operations.c`

```
/*
 * Program : Matrix Operations using Dynamic Memory Allocation
 * Author  : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 *   Dynamically allocates two matrices and performs addition,
 *   subtraction and multiplication using pointer notation only.
 *   Every allocated block is released before the program ends.
 */

#include <stdio.h>
#include <stdlib.h>

/* Function prototypes */
int  **allocateMatrix(int rows, int cols);
void   readMatrix(int **m, int rows, int cols, char name);
void   displayMatrix(int **m, int rows, int cols, const char *title);
int  **addMatrices(int **a, int **b, int rows, int cols);
int  **subtractMatrices(int **a, int **b, int rows, int cols);
int  **multiplyMatrices(int **a, int **b, int r1, int c1, int c2);
void   freeMatrix(int **m, int rows);

int main(void)
{
    int r1, c1, r2, c2;
    int **A = NULL, **B = NULL;
    int **sum = NULL, **diff = NULL, **prod = NULL;

    printf("=== Matrix Operations (Dynamic Memory) ===\n\n");

    /* ---- Read dimensions ---- */
    printf("Enter rows and columns of Matrix A: ");
    if (scanf("%d %d", &r1, &c1) != 2 || r1 <= 0 || c1 <= 0) {
        printf("Invalid dimensions for Matrix A.\n");
        return 1;
    }
    printf("Enter rows and columns of Matrix B: ");
    if (scanf("%d %d", &r2, &c2) != 2 || r2 <= 0 || c2 <= 0) {
        printf("Invalid dimensions for Matrix B.\n");
        return 1;
    }

    /* ---- Allocate and read matrices ---- */
    A = allocateMatrix(r1, c1);
    B = allocateMatrix(r2, c2);
    if (A == NULL || B == NULL) {
        printf("Memory allocation failed.\n");
        freeMatrix(A, r1);
        freeMatrix(B, r2);
        return 1;
    }

    readMatrix(A, r1, c1, 'A');
    readMatrix(B, r2, c2, 'B');

    printf("\n");
    displayMatrix(A, r1, c1, "Matrix A");
    displayMatrix(B, r2, c2, "Matrix B");

    /* ---- Addition and subtraction: dimensions must be identical ---- */
    if (r1 == r2 && c1 == c2) {
        sum  = addMatrices(A, B, r1, c1);
```

```
        diff = subtractMatrices(A, B, r1, c1);

        if (sum != NULL)
            displayMatrix(sum, r1, c1, "A + B");
        if (diff != NULL)
            displayMatrix(diff, r1, c1, "A - B");
    } else {
        printf("Addition and Subtraction not possible: "
               "both matrices must have the same order.\n\n");
    }

    /* ---- Multiplication: columns of A must equal rows of B ---- */
    if (c1 == r2) {
        prod = multiplyMatrices(A, B, r1, c1, c2);
        if (prod != NULL)
            displayMatrix(prod, r1, c2, "A x B");
    } else {
        printf("Multiplication not possible: "
               "columns of A must equal rows of B.\n\n");
    }

    /* ---- Release all allocated memory ---- */
    freeMatrix(A, r1);
    freeMatrix(B, r2);
    if (sum  != NULL) freeMatrix(sum,  r1);
    if (diff != NULL) freeMatrix(diff, r1);
    if (prod != NULL) freeMatrix(prod, r1);

    printf("All dynamically allocated memory has been released.\n");
    return 0;
}

/* Allocates a rows x cols matrix; returns NULL on failure */
int **allocateMatrix(int rows, int cols)
{
    int i, j;
    int **m = (int **)malloc(rows * sizeof(int *));
    if (m == NULL)
        return NULL;

    for (i = 0; i < rows; i++) {
        *(m + i) = (int *)calloc(cols, sizeof(int));
        if (*(m + i) == NULL) {
            for (j = 0; j < i; j++)       /* undo partial allocation */
                free(*(m + j));
            free(m);
            return NULL;
        }
    }
    return m;
}

/* Reads matrix elements from the user */
void readMatrix(int **m, int rows, int cols, char name)
{
    int i, j;
    printf("Enter %d elements of Matrix %c (row-wise):\n", rows * cols, name);
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", (*(m + i) + j));
}

/* Prints a matrix with a heading */
void displayMatrix(int **m, int rows, int cols, const char *title)
{
    int i, j;
    printf("%s:\n", title);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++)
```

```
            printf("%6d", *(*(m + i) + j));
        printf("\n");
    }
    printf("\n");
}

/* Returns a new matrix holding A + B */
int **addMatrices(int **a, int **b, int rows, int cols)
{
    int i, j;
    int **res = allocateMatrix(rows, cols);
    if (res == NULL)
        return NULL;

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            *(*(res + i) + j) = *(*(a + i) + j) + *(*(b + i) + j);
    return res;
}

/* Returns a new matrix holding A - B */
int **subtractMatrices(int **a, int **b, int rows, int cols)
{
    int i, j;
    int **res = allocateMatrix(rows, cols);
    if (res == NULL)
        return NULL;

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            *(*(res + i) + j) = *(*(a + i) + j) - *(*(b + i) + j);
    return res;
}

/* Returns a new (r1 x c2) matrix holding A x B */
int **multiplyMatrices(int **a, int **b, int r1, int c1, int c2)
{
    int i, j, k;
    int **res = allocateMatrix(r1, c2);
    if (res == NULL)
        return NULL;

    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++)
            for (k = 0; k < c1; k++)
                *(*(res + i) + j) += *(*(a + i) + k) * *(*(b + k) + j);
    return res;
}

/* Frees every row, then the array of row pointers */
void freeMatrix(int **m, int rows)
{
    int i;
    if (m == NULL)
        return;
    for (i = 0; i < rows; i++)
        free(*(m + i));
    free(m);
}
```

7. Sample Output

Test Case 1: Same order matrices (2 x 3 and 2 x 3)

```
=== Matrix Operations (Dynamic Memory) ===

Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 2 3
Enter 6 elements of Matrix A (row-wise):
1 2 3 4 5 6
Enter 6 elements of Matrix B (row-wise):
7 8 9 1 2 3

Matrix A:
     1     2     3
     4     5     6

Matrix B:
     7     8     9
     1     2     3

A + B:
     8    10    12
     5     7     9

A - B:
    -6    -6    -6
     3     3     3

Multiplication not possible: columns of A must equal rows of B.

All dynamically allocated memory has been released.
```

Test Case 2: Compatible for multiplication (2 x 3 and 3 x 2)

```
=== Matrix Operations (Dynamic Memory) ===

Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 3 2
Enter 6 elements of Matrix A (row-wise):
1 2 3 4 5 6
Enter 6 elements of Matrix B (row-wise):
7 8 9 10 11 12

Matrix A:
     1     2     3
     4     5     6

Matrix B:
     7     8
     9    10
    11    12

Addition and Subtraction not possible: both matrices must have the same order.

A x B:
    58    64
   139   154

All dynamically allocated memory has been released.
```

Test Case 3: Square matrices (3 x 3 and 3 x 3), all three operations

```
Enter rows and columns of Matrix A: 3 3
Enter rows and columns of Matrix B: 3 3
  (A = 1 2 3 / 4 5 6 / 7 8 9    B = 9 8 7 / 6 5 4 / 3 2 1)

A + B:
    10    10    10
    10    10    10
    10    10    10

A - B:
    -8    -6    -4
    -2     0     2
     4     6     8

A x B:
    30    24    18
    84    69    54
   138   114    90
```

8. Explanation of the Program

- **Memory allocation:** allocateMatrix() uses malloc() for the array of row pointers and calloc() for each row, so every element starts at zero. If any allocation fails, the rows already allocated are freed and NULL is returned.
- **Pointer notation:** every element access, in input, display, addition, subtraction and multiplication, uses *(*(m + i) + j) rather than m[i][j].
- **Validation:** the program checks the orders before each operation, so it never reads or writes outside the allocated memory.
- **Separate functions:** addMatrices(), subtractMatrices() and multiplyMatrices() each create and return their own result matrix, which keeps main() short and readable.
- **Memory release:** freeMatrix() frees each row first and then the row-pointer array. main() calls it for A, B and each result matrix that was created.

9. Conclusion

The program successfully allocates two matrices at run time and performs addition, subtraction and multiplication using pointer notation, with each operation written as a separate function. Input orders are validated before every operation, and all dynamically allocated memory is released before the program terminates. The program was compiled with gcc using the -Wall -Wextra flags without warnings, and tested with AddressSanitizer, which reported no memory errors or leaks.